

Gate-Pinned State Memory

Exact, zero-context memory in a model's recurrent state, served by tang

DRAFT

CAM PEDERSEN · MUNICIPAL ROBOTICS

Abstract

A coding agent remembers by pasting text into its prompt, and pays for it on every turn. Divyansh Agrawal's qwenputer showed another way: a Gated DeltaNet layer with its gates pinned is exact, addressable RAM, and a filesystem can be compiled into extra layers of a model without training[1]. We checked every link of that claim on our own stack.

tang's kernel stores and reads one-hot rows bitwise, and stays exact over a 50,000-step free run. Trained heads cannot do it: on GLM-5.3-Flash, none of 8,192 channels per layer keeps its gate at exactly 1. We reproduced the author's filesystem on an RTX 3090, with the model issuing every store call itself and recalling files from a 14 KB state file in an empty conversation. tang's reference math then ran the author's compiled layers bitwise, tang's own Rust compiler emitted all 168 tensors of them bitwise, and a CUDA window in our engine ran them bitwise at about 0.9 ms per token, 25 to 40 times faster than the author's stack.

We also tested two ways to get instant prefill from the same structure, and both fail. What remains we call Gate-Pinned State Memory (GPSM): memory that costs no context, needs no training, leaves the model's text unchanged, and can be held by the user rather than the provider.

DRAFT

1 Motivation

A coding agent's memory today is text. Notes, summaries and retrieved chunks go into the prompt, are prefilled on every turn, and compete with code for the context window. On a 16 GB laptop running a 4B model, context is the scarcest thing there is. The alternatives in the literature each pay somewhere else. KV caches are exact but large and tied to one model[2]. Learned fixed-size states cost no context but forget[3,4]. Test-time training writes facts into weights, at the price of optimizer state and lost general ability[5].

Gated DeltaNet (GDN) updates a matrix state per head with the delta rule[6]:

$$S_t = g_t S_{t-1} + \beta_t k_t (v_t - S_{t-1}^{\top} k_t)^{\top}$$

With the decay $g = 1$, the write strength $\beta = 1$ and a one-hot key $k = e_i$, the update replaces row i of S with v and leaves every other row alone.



FIG 1 The delta rule, drawn. Sixteen rows stand for a 128 × 128 head state. A trained head decays every row and smears the write across all of them. A pinned head with a one-hot key replaces exactly one row and leaves the rest untouched.

A one-hot query reads row i back. Compile weights that produce those gates and keys, and the recurrent state is RAM. We call fixing the gates this way *gate pinning*, a head built this way a *pinned head*, and memory kept in pinned heads **Gate-Pinned State Memory** (GPSM). The gates are pinned, not the state: rows change when they are rewritten, and stay exactly as written until then. qwenputer compiled a RISC-V CPU and a 512-file store into 12 extra layers in front of Qwen 3.5 4B, without training, and reported a KL divergence to the base model of 6×10^{-4}[1].

tang is our inference engine. It runs Qwen3-Coder-Next, a hybrid of 36 GDN layers and 12 attention layers, and already snapshots the fixed-size recurrent state to disk. frog, our coding agent, talks to it. So the question was concrete: what of this transfers to an engine we own?

2 Exact RAM in tang's kernel

tang pins the GDN recurrence in a CPU specification that the CUDA kernel is verified against bitwise. We tested the claim there.

- **The gates reach exactly 1.** Projections of $a + \mathrm{dt} = -30$ and $b = 40$ round to exactly `1.0f`.

DRAFT

- **A write into an empty row stores v bitwise** and touches no other row of any of the 48 heads.
- **An overwrite stores** $\mathrm{fma}(1, v - s, s)$, which is exactly v only when $v - s$ is exact. On random floats, 8,643 overwrites were exact and 15,933 were off by one rounding. Stored values must be integers below 2^{24}.
- **A 50,000-token free run**, driving the kernel as a register machine, stayed bitwise exact on the CPU and on an RTX 3090.
- **One token with $g < 1$ ends exactness for good.** It scales every row by a non-power-of-two, and the integers turn fractional.

3 Trained heads cannot do it

So a pretrained head can be borrowed for storage only if its gate is bitwise 1 on every token. We measured the real gates of GLM-5.3-Flash, whose recurrent layers use a per-channel-gated delta rule, over 427 tokens and all 34 recurrent layers.

TABLE 1 Gates of a trained hybrid model. No channel qualifies as exact storage. Measured on GLM-5.3-Flash UD-Q4_K_XL, RTX 3090.

LAYERS	MEAN G	BEST CHANNEL MIN G	B MEAN / MAX	AT 1.0 EVERY TOKEN
Early (0–10)	0.51–0.68	0.999994	0.54–0.71 / ≤0.997	0 of 8,192
Mid (12–26)	0.73–0.87	0.999999	0.64–0.77 / ≤1.000	0 of 8,192
Late (28–44)	0.67–0.79	0.999997	0.55–0.84 / 1.000	0 of 8,192

A few channels are remarkably slow. Between 1 and 8 percent per layer have a mean gate above 0.999, and the best has a gate-only half-life near three million tokens.

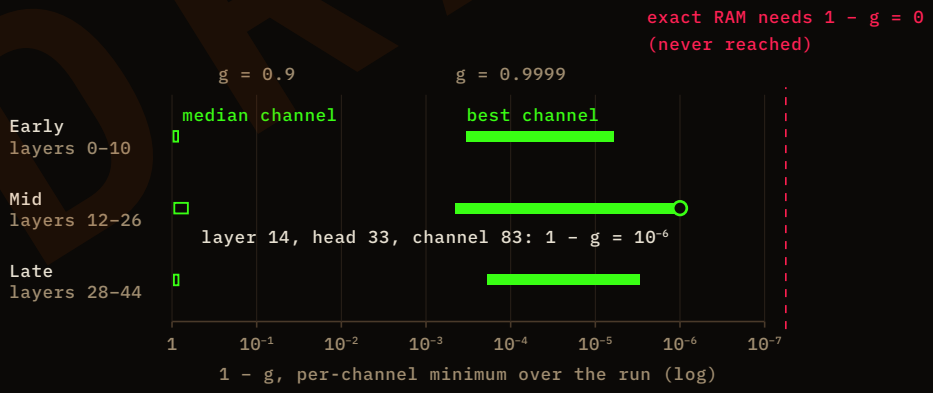


FIG 2 No channel reaches 1. Each bar is a range across the layers in its group: hollow for the median channel's minimum gate over the run, solid for the best channel's. GLM-5.3-Flash on an RTX 3090 over 427 tokens; per-layer data was not kept.

But they are analog registers, and they leak. Exact storage needs dedicated pinned heads with compiled gates, which is the qwenputer design and the basis of GPSM.

DRAFT

4 Two dead ends

The same structure suggests faster prefill. We tested both versions, and neither works.

Poking state. A trained head can only read the state its own prefill would have produced. An integer written into a row means nothing to it, so poking context into state turns back into prefill.

Chunked kernels. tang steps the GDN recurrence one token at a time. A chunked tensor-core kernel has about 35 times the throughput on a 512-token window, but the recurrence is only 4.5 percent of a prefill window on an RTX 3090, and 5.5 percent on an H200. Prefill time lives in attention and the MoE layers.

Precomputed chunk maps. The delta rule is affine in S, so a chunk of text collapses to one matrix pair per head. If a chunk's keys and values did not depend on earlier context, prefill would be a few matrix products. We spliced context-free chunks into real GLM-5.3-Flash runs.

TABLE 2 Context-free chunk precomputation. KL is over the 32 tokens after each spliced chunk. About 0.1 nat per token: better than skipping the chunk, not lossless.

SPLICED CHUNK	STATE ERROR, EARLY / MID / LATE	KL, MEAN / MAX (NATS)	TOP-1 AGREE
4	0.34 / 0.65 / 0.54	0.095 / 0.80	30 / 32
8	0.30 / 0.62 / 0.46	0.103 / 1.13	27 / 32
12	0.44 / 0.98 / 0.60	0.111 / 0.89	31 / 32
Skip (control)	0.5–0.7	0.30–0.58 / 3.7–5.5	26–28 / 32

Mid and late layers depend on context, not position. Fixing the error means recomputing most of the depth, which is prefill again. Instant restore of *old* context is what tang already ships: a resumed 7,000-token session prefills in about 0.1 s from the snapshot disk tier.

5 GPSM, end to end

5.1 An engine-driven store

We first built the store without compiled layers: a key-value file store in dedicated GDN heads of one layer, written and read through the real kernel with one-hot keys and pinned gates. The layout follows the author's: a hashed directory with linear probing, plus content rows of token ids. Reads pass the output norm and a division, so values stay below 2^{18}.

TABLE 3 Store capacity in a quarter of one GDN layer.

MODEL SHAPE	HEADS, K / V	DIRECTORY	CONTENT
Flash-Next	16 / 48, tiled	1,024 slots, 512 files	65,024 tokens · 768 KiB
Coder-Next, Qwen 3.5 4B	16 / 32, grouped	512 slots, 256 files	65,024 tokens · 512 KiB

DRAFT

A 512-file fill and a 10,000-operation random workload, checked against a hash map, both pass. The store persists through the bytes tang's snapshot already carries. It also moves into a different conversation's state while leaving that conversation's own heads bitwise untouched.

5.2 The model drives it

Reproduction. We ran the author's code and checkpoint on an RTX 3090. In one conversation the model issued all nine store calls itself: three puts, a list, three gets, a delete and a list. Every answer was printed by the model, and 160 printed tokens matched the author's reference with no mismatches. We saved the state to a 14 KB file and loaded it into an empty conversation. The model listed the two surviving files and read both back exactly. KL to the base model was 6.9×10^{-4} against a noise floor of 6.7×10^{-4}, and bitwise identical with files loaded or not.

tang runs the compiled layers. tang's reference math ran the checkpoint's 12 compiled layers on the author's trace. All 126 tokens matched, and every store head's 128×128 state was bitwise equal after each operation.

tang compiles them. `tang-chip`, a Rust port of the author's compiler, emits the layers from the circuit graph: 3,753 dimensions, 164 heads and 47,532 units. At Qwen 3.5 4B shapes, all 168 tensors equal the checkpoint bitwise, zero padding included: 2.44 billion elements, 1.24 million of them nonzero. Retargeted to Coder-Next, whose hidden size widens from 2,048 to 3,968, the store replays the trace bitwise and no weight touches the host model's dimensions.

tang serves them. A chip window in the engine uploads the emitted layers as sparse blocks and runs store calls without touching the base layers, the LM head or the KV cache. On an RTX 3090 it ran with 0 mismatching layer steps against the CPU path. Inside a running engine, on a slot holding a prompt, store calls returned the stored ids. The slot's tokens and greedy continuation stayed identical to a twin slot that never ran them.

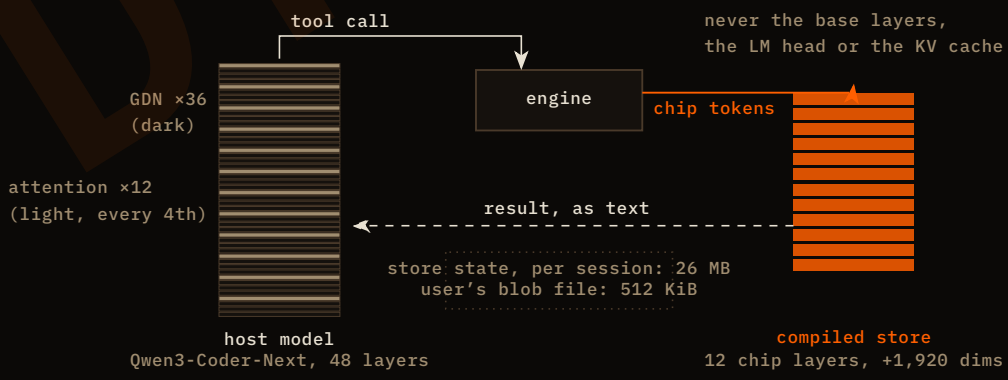


FIG 3 Where the store lives. The compiled layers sit beside the host model, not inside it. Store calls run only through the chip layers; the host sees the result as text.

DRAFT

TABLE 4 **The chip window on an RTX 3090**, with CUDA graphs. The author's PyTorch stack took 1.0–1.65 s per operation.

OPERATION	TOKENS	PER OP	PER TOKEN
Put	56	51 ms	0.92 ms
Get	43	40 ms	0.92 ms
List	27	25 ms	0.93 ms

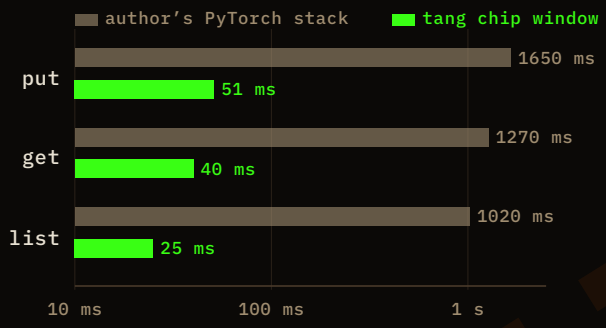


FIG 4 **Latency per store operation**, same RTX 3090, log scale. The author's time is mostly PyTorch fallback kernels, not the store.

The compiled layers take 13.75 MB of sparse weights and 26 MB of state per session. They cost about 332 million multiply-adds per token, roughly 8 percent of the 4B model.

Each link of the claim was checked on its own, from the kernel up to a running engine.

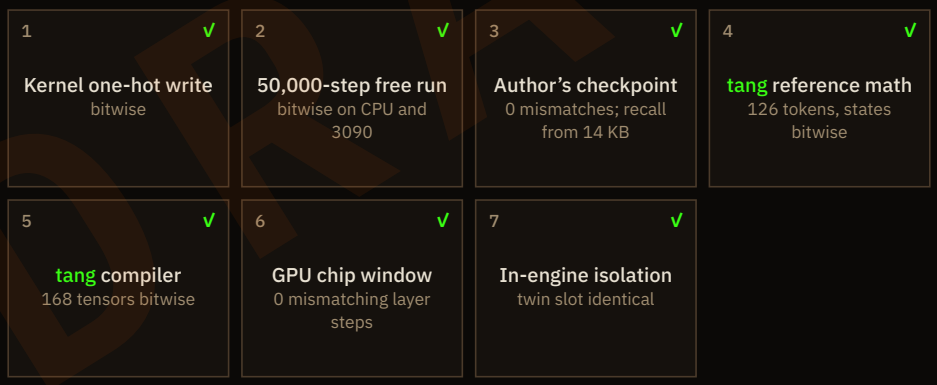


FIG 5 **The proof chain.** Seven checks, each bitwise or exact, from the kernel's one-hot write to a store call inside a running engine.

5.3 Serving

tang serve loads a store per session, runs operations and exports it. An ephemeral mode zeroes the store when a session ends and never writes it to disk. frog loads the user's file at session start, saves after every write, and gives the model a `Memory` tool. `frog memory dump` prints exactly what is stored.

DRAFT

> ASIDE · PRIVACY
> The part of memory that persists becomes a file on the user's disk, about half a megabyte. In a hosted session the provider sees it in plaintext while the session runs, and with the ephemeral mode keeps nothing afterwards. This is not encryption: the layout is known, so anyone holding the file can read it. The model can still repeat what it reads, so egress controls matter as before. What changes is where memory piles up: in a file you can read and delete, not with us.

6 Where it sits

GPSM occupies a corner nothing else holds: exact recall, no context, no training, and almost no change to the base model. Text memory such as MemGPT, Mem0 and A-MEM is portable and mature, but pays context every turn and recalls lossily[7–9]. KV reuse is exact but large[2]. Learned states and test-time training cost no context but are lossy and need training[4,10,11]. Knowledge editing and one-step LoRA memory are dominated: one study measured free-form recall at zero[5]. Compiled transformers go back to Tracr[12]. What is new is that hybrid models have layers where compiled storage is exact.

7 What is not done

- **No cross-session benchmark.** The 90 percent recall figure is the author's own fact set. Nobody has run GPSM, or the author's store, on LongMemEval or LoCoMo.
- **Does the model reach for it?** The 4B model used the store when asked. Whether Coder-Next uses memory at the right moments in a long coding session is unmeasured, and it is the main risk.
- **No build on real Coder-Next weights yet.** It needs the circuit re-exported with Coder-Next's tokenizer and a machine that holds the weights.
- **Fixed capacity.** 256 to 512 files.
- **Compute on every token.** About 8 percent at 4B, less on a 3B-active mixture of experts.

References

1. Agrawal, D. Qwenputer: a fully functioning, deterministic RISC-V CPU inside Qwen3.5. (2026).
2. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. https://arxiv.org/abs/2405.16444 (2024).
3. Stuffed Mamba: Oversized States Lead to the Inability to Forget. https://arxiv.org/abs/2410.07145 (2024).
4. Titans: Learning to Memorize at Test Time. https://arxiv.org/abs/2501.00663 (2025).
5. Beyond Perplexity: A Behavioral Evaluation Framework for Deployment-Memory Claims in LLM Test-Time Training. https://arxiv.org/abs/2607.00368 (2026).

DRAFT

6. Gated Delta Networks: Improving Mamba2 with Delta Rule. https://arxiv.org/abs/2412.06464 (2024).
7. MemGPT: Towards LLMs as Operating Systems. https://arxiv.org/abs/2310.08560 (2023).
8. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. https://arxiv.org/abs/2504.19413 (2025).
9. A-MEM: Agentic Memory for LLM Agents. https://arxiv.org/abs/2502.12110 (2025).
10. In-Place Test-Time Training. https://arxiv.org/abs/2604.06169 (2026).
11. Latent Context Compilation: Distilling Long Context into Compact Portable Memory. https://arxiv.org/abs/2602.21221 (2026).
12. Tracr: Compiled Transformers as a Laboratory for Interpretability. https://arxiv.org/abs/2301.05062 (2023).

DRAFT